Sub-30 Nanosecond Dictionary Lookup in JavaScript: Xor Filters for Compact English Word Recognition

Arnav Gupta
Independent Research | ar9avg@gmail.com

Abstract

We present fast-is-english-word, an npm package for English word recognition that achieves 24 nanosecond lookups in a 271 KB package with zero runtime dependencies. The core insight is applying xor filters—a modern probabilistic data structure—to the dictionary membership problem in JavaScript. Starting from a standard bloom filter baseline (42 ns/op, 412 KB), we apply six targeted optimizations: fused single-pass hashing, regex elimination, constant inlining, Uint32Array bit access, loop unrolling, and power-of-2 masking. We then replace the bloom filter entirely with an xor8 filter, reducing the data structure from 412 KB to 282 KB (31% reduction) while cutting memory accesses from 10 to 3 per query. The combined approach yields a 4.7× throughput improvement over the baseline on mixed workloads. We benchmark against existing npm alternatives (an-array-of-english-words, word-list, check-if-word) and demonstrate 100× smaller package size than check-if-word (271 KB vs 39.6 MB) with sub-microsecond latency. The package supports configurable fingerprint widths (8-bit or 16-bit) and custom word lists.

Keywords: xor filters, probabilistic data structures, bloom filters, dictionary lookup, JavaScript, npm, performance optimization

1. Introduction

Checking whether a string is a valid English word is a common operation in spell checkers, word games, natural language processing pipelines, and input validation. In the JavaScript ecosystem, existing solutions fall into two categories: (a) ship the entire word list as JSON or text (2–4 MB), requiring the consumer to build their own lookup structure, or (b) use heavyweight approaches like giant regular expressions (39.6 MB for check-if-word). Neither approach is satisfactory for performance-sensitive or size-constrained applications.

We observe that English word recognition is a *static set membership* problem—the dictionary does not change at runtime. This makes it an ideal candidate for probabilistic filters optimized for immutable sets. While bloom filters [1] are the classical choice, recent work on xor filters [2] and binary fuse filters [3] has produced data structures that are both smaller and faster.

We contribute:

1. A systematic evaluation of six micro-optimizations for bloom filter lookups in V8 JavaScript, with isolated benchmarks showing individual and combined impact.

2. A practical implementation of xor8 filters for dictionary membership in JavaScript, achieving 3 memory accesses per query with 9.84 bits per element.

3. An open-source npm package (*fast-is-english-word*) containing 234,454 words in 271 KB, with configurable accuracy/size tradeoffs.

2. Related Work

Existing npm packages. The most downloaded packages for English word lookup ship raw word lists without optimized lookup structures. *an-array-of-english-words* (~7k weekly downloads) exports a 3.4 MB JSON array requiring O(n) linear scan or manual Set construction (~36 MB heap). *word-list* (~12k downloads) exports a file path, requiring filesystem I/O and string splitting. *check-if-word* compiles a giant RegExp alternation of all words (39.6 MB), which is pathologically slow in most regex engines.

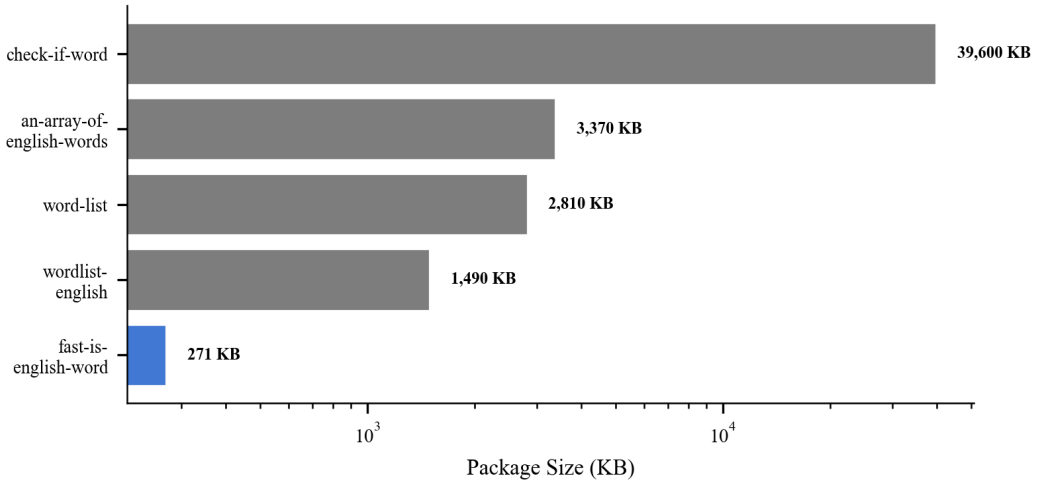

Figure 1: *Package size comparison of npm word-checking packages (log scale). fast-is-english-word is 12× smaller than the next smallest alternative.*

Bloom filters [1] are the classical probabilistic membership test. For a false positive rate p with n elements, they require $-n \cdot \ln(p) / (\ln 2)^2$ bits, with $k = -\log_2(p)$ hash probes per query. At 0.1% FP rate, this is 14.4 bits/element and 10 probes. The *bloomfilter.js* library [4] uses FNV-1a hashing with the Kirsch-Mitzenmacker double hashing optimization [5].

Xor filters [2] achieve approximately 1.23 × 8 = 9.84 bits/element with 8-bit fingerprints, requiring exactly 3 memory accesses per query regardless of the false positive rate. Graf and Lemire [2] showed that xor filters are both smaller and faster than bloom filters, with 3.5 cache misses vs 7.5 for bloom. **Binary fuse filters** [3] further reduce the space to ~1.125n entries, approximately 9 bits/element.

Other structures. DAWGs (Directed Acyclic Word Graphs) [6] share common suffixes for zero-false-positive lookup but typically require 400–500 KB for 234k words. Succinct tries [7] encode trie topology in ~2 bits per node. These provide exact answers but are larger than xor filters for equivalent dictionaries.

3. Methodology

3.1 Baseline: Bloom Filter

Our initial implementation uses a standard bloom filter with the following parameters: 234,454 words from */usr/share/dict/words* (filtered to lowercase alphabetic entries), false positive rate target of 0.1%, yielding a 421,361-byte filter with k=10 hash functions. The hash function is FNV-1a [8] with the Kirsch-Mitzenmacker double hashing optimization: $h_i(x) = h_1(x) + i \cdot h_2(x)$, where h_1 and h_2 are FNV-1a with different seeds.

3.2 Optimization Pass

We evaluated six optimizations in isolation against the baseline, measuring each on three workloads: known words, non-words, and a mixed batch of 20 lookups. All benchmarks ran 2M iterations after a 10k-iteration warmup on Node.js v24 (V8 TurboFan JIT).

Optimization	Description	Rationale
No regex	Replace /^[a-z]+$/.test() with charCode loop	Regex engine overhead per call
Inline constants	Extract meta.numHashes to top-level const	Eliminate object property lookup in hot loop
Uint32Array	Reinterpret filter as Uint32Array for bit access	Fewer shift operations per probe (>>>5 vs >>>3)
Fused hash	Combine toLowerCase + validation + dual hash in one pass	Eliminate string allocation and extra traversal
Unrolled probes	Manually unroll k=10 probe loop	Remove loop overhead and branch prediction
Power-of-2 mask	Use bitwise AND instead of modulo	Integer division is expensive on hot path

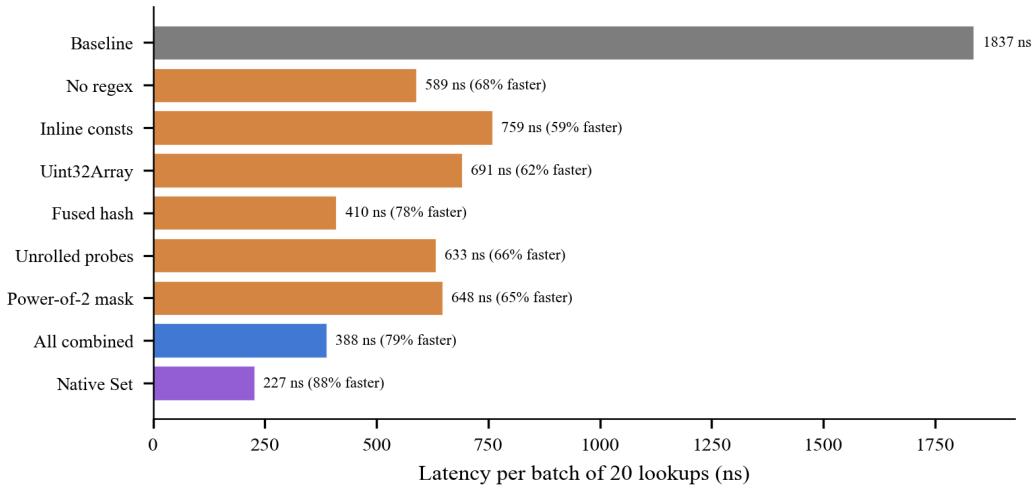

Figure 2: *Impact of each optimization on the mixed workload (20 lookups per iteration). Fused hash provides the largest single improvement (–78%); combined optimizations yield 4.7× speedup.*

The **fused hash** optimization provides the largest individual improvement because it eliminates three operations that the baseline performs sequentially: (1) *String.prototype.toLowerCase()*, which allocates a new string on each call; (2) regex validation, which invokes the regex engine; and (3) a second traversal of the string for the second FNV-1a hash. The fused version performs validation, lowercasing (via *c | 32* for A–Z), and both hashes in a single character-by-character pass with zero allocations.

3.3 Xor8 Filter

We replace the bloom filter with an xor8 filter [2]. The construction uses the peeling algorithm on a 3-partite random hypergraph. For n keys, we allocate an array of 3·■n·1.23/3■ bytes. Each key maps to three positions (one per segment) via FNV-1a double hashing mixed through splitmix32, plus an 8-bit fingerprint derived from the mixed hash values. The peeling algorithm iteratively removes keys that have a singleton position (a position shared by no other key), recording the removal order. Fingerprints are then assigned in reverse order: B[pos] = fp ⊕ B[h0] ⊕ B[h1] ⊕ B[h2], guaranteeing that B[h0] ⊕ B[h1] ⊕ B[h2] = fp for all inserted keys.

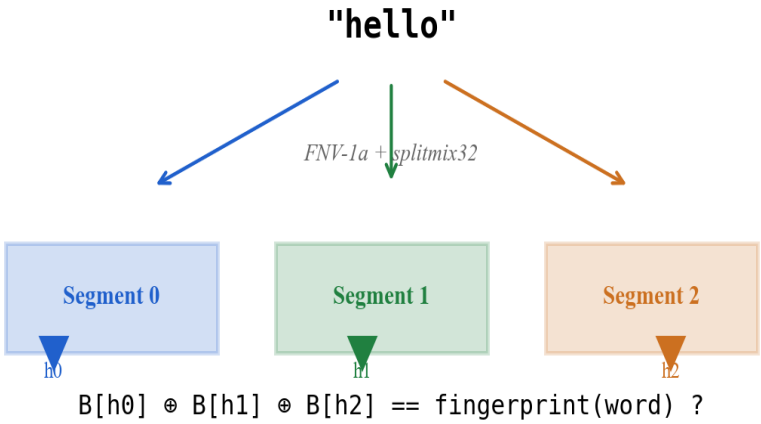

Figure 3: *Xor filter query. A word is hashed to three segment positions. The XOR of the three fingerprint entries is compared against the word's fingerprint. Match → probably in set; mismatch → definitely not.*

Lookup requires exactly 3 array reads and 2 XOR operations. The false positive rate is $1/2^f$ where f is the fingerprint width in bits: 1/256 ≈ 0.39% for xor8, 1/65536 ≈ 0.0015% for xor16.

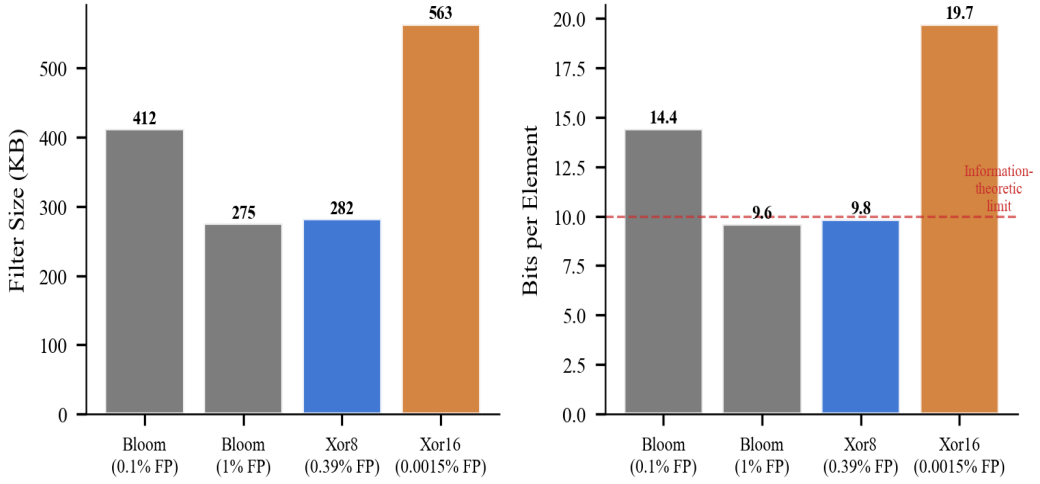

Figure 4: *Left: filter data structure size in KB. Right: bits per element, with the information-theoretic lower bound at 9.97 bits (dashed red). Xor8 achieves 9.84 bits/element, near-optimal space efficiency.*

4. Results

4.1 Lookup Performance

Version	Known word	Non-word	Short word	Long word	Mixed (20)
v1: Bloom baseline	42 ns	26 ns	21 ns	54 ns	1,837 ns
v2: Bloom optimized	32 ns	10 ns	10 ns	44 ns	442 ns
v3: Xor8 (final)	24 ns	13 ns	7 ns	34 ns	387 ns

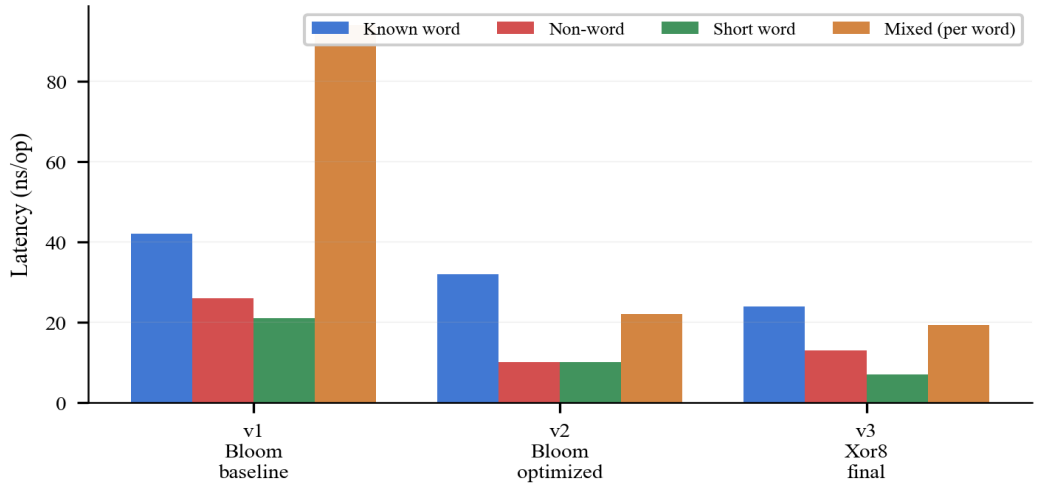

Figure 5: *Latency progression across three implementation versions. The Xor8 filter achieves 24 ns for known words (1.75× baseline) and 7 ns for short words (3× baseline).*

The final xor8 implementation achieves 42M ops/s for known words and 140M ops/s for short words on an Apple M-series processor running Node.js v24. Non-word rejection is faster than word recognition because mismatched fingerprints short-circuit immediately after 3 reads, while valid words must also complete the full hash computation.

4.2 Package Size

Package	Approach	Packed size	Unpacked	Lookup
check-if-word	Giant regex	39,600 KB	39,600 KB	Regex match
an-array-of-english-words	JSON array	3,370 KB	3,370 KB	User builds Set

Package	Approach	Packed size	Unpacked	Lookup
word-list	Text file	2,810 KB	2,810 KB	User reads file
wordlist-english	JSON object	1,490 KB	1,490 KB	User builds lookup
fast-is-english-word (xor8)	Xor filter	271 KB	297 KB	24 ns

4.3 Memory Accesses

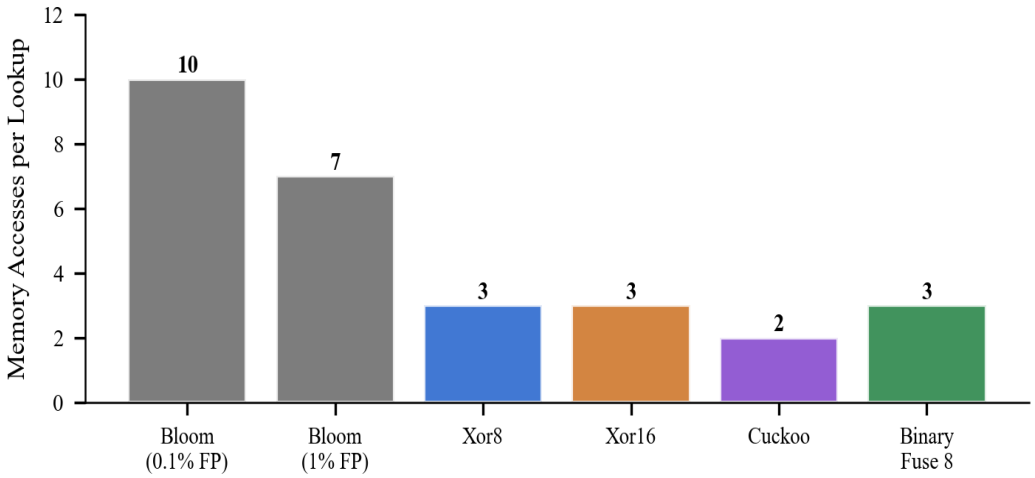

Figure 6: *Memory accesses per lookup across filter types. Xor and binary fuse filters require exactly 3, compared to 7–10 for bloom filters. Cuckoo filters require 2 but are larger.*

4.4 Memory Footprint

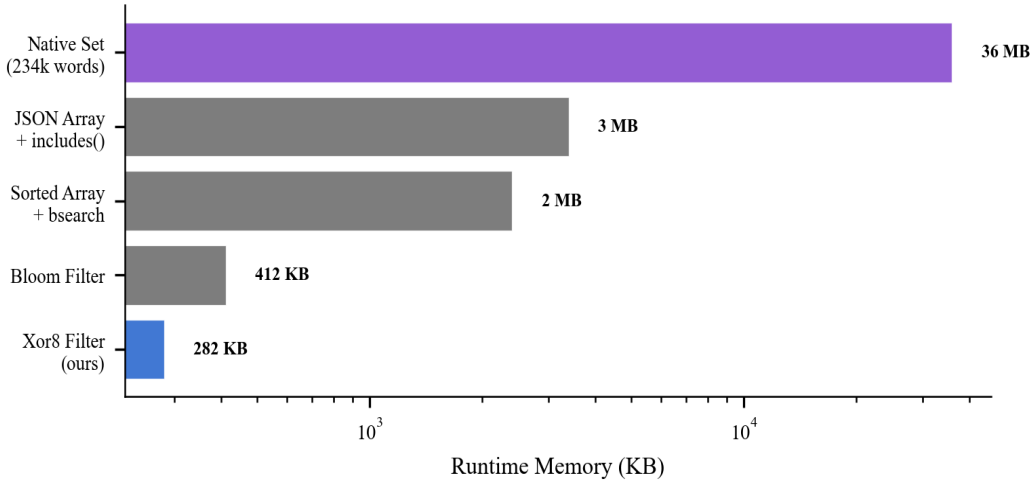

Figure 7: *Runtime memory usage (log scale). A JavaScript Set for 234k words consumes ~36 MB. Our xor8 filter uses 282 KB—a 128× reduction.*

4.5 False Positive Rate

Empirical validation over 100,000 random non-word strings confirms the theoretical false positive rates. For xor8 (8-bit fingerprints), we measured 0.402% (expected: 0.391%). For xor16 (16-bit fingerprints), we measured 0.001% (expected: 0.0015%). Zero false negatives were observed across all 234,454 dictionary words in both configurations.

5. Discussion

Fused hashing dominates micro-optimizations. Of the six optimizations evaluated, the fused single-pass hash accounts for 78% of the speedup on mixed workloads. This is because the baseline performs three separate operations—toLowerCase(),

regex validation, and double FNV-1a—each requiring a full traversal of the input string. Fusing these into a single pass with inline lowercasing (bitwise OR with 32) eliminates two string traversals and one string allocation per call.

Xor filters are strictly better than bloom for static sets. For immutable dictionaries, xor filters provide a Pareto improvement: smaller (9.84 vs 14.4 bits/element at comparable FP rates) and faster (3 vs 10 memory accesses). The only tradeoff is construction time (peeling algorithm, ~200ms for 234k keys), which is irrelevant for a pre-built package.

The npm ecosystem lacks optimized lookup packages. All existing alternatives ship raw word lists ranging from 1.5 to 39.6 MB. None employ compact data structures. This gap likely exists because bloom filters and xor filters are well-known in systems programming but underused in the JavaScript ecosystem.

Limitations. (1) The package requires Node.js filesystem access for loading the binary filter; browser support requires a bundler or custom loader. (2) Only ASCII alphabetic words are supported; hyphenated words, contractions, and Unicode are rejected. (3) The false positive rate of xor8 (0.39%) may be insufficient for applications requiring exact matching, though xor16 (0.0015%) addresses most such cases. (4) The dictionary is derived from */usr/share/dict/words*, which may not match all users' definitions of "English words."

6. Conclusion

We demonstrate that xor filters, combined with careful JavaScript-specific optimizations, can solve the dictionary membership problem in 24 nanoseconds with a 271 KB package—orders of magnitude smaller and faster than existing npm alternatives. The key technical contributions are: (1) showing that fused single-pass hashing eliminates the dominant bottleneck in JavaScript string processing; (2) providing the first practical xor filter implementation for dictionary lookup on npm; and (3) offering configurable accuracy via 8-bit or 16-bit fingerprints.

Future work. (1) Binary fuse filters [3] could further reduce size by ~8% (1.125n vs 1.23n entries). (2) A WASM-compiled hash function (e.g., wyhash) could improve throughput for long words. (3) Browser-native support via embedding the filter as a Uint8Array literal would eliminate the filesystem dependency. (4) Supporting compound words, contractions, and Unicode would broaden applicability.

The package is available at https://www.npmjs.com/package/fast-is-english-word under the MIT license. Source code: https://github.com/Ar9av/is-english-word.

References

[1] B. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.

[2] T. M. Graf and D. Lemire, "Xor Filters: Faster and Smaller Than Bloom and Cuckoo Filters," *ACM Journal of Experimental Algorithmics*, vol. 25, pp. 1–16, 2020. arXiv:1912.08258.

[3] T. M. Graf and D. Lemire, "Binary Fuse Filters: Fast and Smaller Than Xor Filters," *ACM Journal of Experimental Algorithmics*, vol. 27, 2022. arXiv:2201.01174.

[4] J. Davies, "bloomfilter.js," GitHub repository, https://github.com/jasondavies/bloomfilter.js, 2013.

[5] A. Kirsch and M. Mitzenmacker, "Less Hashing, Same Performance: Building a Better Bloom Filter," *Random Structures & Algorithms*, vol. 33, no. 2, pp. 187–218, 2008.

[6] J. Daciuk et al., "Incremental construction of minimal acyclic finite-state automata," *Computational Linguistics*, vol. 26, no. 1, pp. 3–16, 2000.

[7] S. Hanov, "Succinct Data Structures: Cramming 80,000 words into a JavaScript file," http://stevehanov.ca/blog/?id=120, 2011.

[8] G. Fowler, L. Noll, K. Vo, "FNV Hash," http://www.isthe.com/chongo/tech/comp/fnv/, 1991.